Evidence and Ledger Audit Against Coding Standards

Prepared for Puma | 22 October 2026 | Read-only findings and proposed clarifications

Executive conclusion

There is substantial recurring rework, but the sampled records do not establish that the current standards directly caused most of it. The dominant pattern is incomplete or late application of principles that already existed: proportionate verification, faithful boundary observation, source-bound evidence, ownership, and one authoritative current disposition.

The strongest standards-side opportunities are narrower: remove broad lower-level wording that can sound more mandatory than Core; make evidence subject attribution precise; explicitly apply lifecycle and environment principles to verification clients; and route proportionality into acceptance-stage work, not just implementation.

Recommendation: make a small clarification/consolidation pass, not a larger rulebook. Improve how existing principles are applied at the deciding boundary. Preserve failed measurements and immutable evidence, while reducing redundant presentations and competing status narratives.

What the evidence supports

High confidence: repeated harness failures, redundant verification, brittle exact-text assertions, ledger duplication, and late boundary/ownership findings occurred. High confidence: the relevant generic rules were already present in adopted standards. Medium confidence: the proposed wording/routing changes would reduce recurrence. Low confidence: those ambiguities caused any particular observed failure.

What this audit does not conclude

Large evidence volume is not proof of wasted effort. A failed candidate followed by a justified fresh experiment is not automatically churn. Review findings that correct physics, ownership, custody or persistence defects are valuable. Tool capacity, permission denials and unavailable runtime facilities are not, by themselves, coding-standard defects.

No standards, repository code, issues, PRs or acceptance criteria were changed. No tests, benchmarks or scientific campaigns were rerun for this audit. Linked records were inspected; their runtime results were not independently reproduced here.

1. Method and historical applicability

Primary case: Rheon PR59 at 4d93d68, including its warm-start ledger, plans, historical results and integration/requalification records. Broader Rheon sample: main 20a0e1e, PR142/138/143 records, and published voxel branch 7438e35. Cross-project comparator: Tuldok PR43 at 51bec9e, PR53 at 5c0e7f9, and scientific-integration main 133d53c. Standards reference: Coding-Standards 73bd032b.

The broader Rheon sample inspected 16 selected authored files, two plan trees and relevant PR discussions. Tuldok was a bounded comparator, not an exhaustive second repository audit. Findings are recurrence examples within these samples, not repository-wide defect rates or estimates of total developer time.

The rules were already present

Adopted standards revisions were established from project records, rather than guessed from dates: PR59 cited 188beda1; the Rheon composition records cited 0fc92a2; Tuldok cited dcc56f26. Some Git timestamps are anomalous, so timestamp-only reconstruction was not treated as reliable evidence of adoption.

Eight fetched standards files were byte-identical across those three adopted revisions and current 73bd032b: Core, Planning, Verification, Development Proportionality, Documentation, Commit, Tooling and Performance. Router and Rust Tooling were additionally identical at the two Rheon revisions and current; those two were not compared at the Tuldok revision.

This materially weakens an explanation that agents lacked a proportionality or evidence-reuse principle at the time. It does not prove agents read, understood or correctly applied every applicable clause.

Contemporaneous PR59 standards pin | Tuldok standards pin | Pinned current Core

Causal categories used

1. Direct rule defect: a mandatory clause requires a harmful outcome. None was established for the sampled downstream incidents.
2. Ambiguity or routing weakness: lower-level wording can plausibly mislead despite stronger principles.
3. Application failure: an existing applicable principle was missed or applied late.
4. Tool/environment failure: independent execution or access limitation.
5. Legitimate discovery: a real defect or changed subject justifies new work.

The standards library also records historical procedural amplification and obsolete-prose checker coupling in its own effectiveness audit. These are useful precedents, but should not be relabeled as unfixed current rules without checking the adopted content.

2. Recurring evidence-production failures

A. Validation machinery created new work

PR59 documents an initially unbalanced four-permutation schedule; diagnostic/production target sharing; an overstated harness-equivalence claim; an inspection command that rewrote its input binaries; and native driver V1-V3 repairs. A reviewer described V1 but finally hashed V2 while edits overlapped. Corrective reports, rebuilt artifacts and further review were then necessary.

These are supporting-tool and attribution failures, not repeated failure of the corrected solver. The subsequent recovery often was justified. In particular, no timing had run when binary inspection mutated inputs, so the later fixed timing cohort was not contaminated.

Mutating inspection and correction | V1 findings / V2 identity race

B. Successful-looking execution did not establish the obligation

One PR59 suite printed 792 passed, 0 failed and 13 ignored, but its outer wrapper returned 1. The suite was repeated to obtain an explicit successful child result; the confirmation took 59.10 seconds. Four separate zero-selection/no-op test invocations are documented across voxel M1, M2, M4 and PR142. They were corrected rather than falsely counted as passing.

The avoidable cause is unreliable selection/outcome capture. Repeating the uncertain suite was a defensible recovery, not evidence that all repetition is waste. Verification must establish that the deciding observation actually executed, not merely that a launcher returned success.

Suite outcome uncertainty | Zero-selection examples | PR142 filter correction

C. Available evidence reuse was applied late

PR142 completed an 18.2-minute local core suite while source-identical hosted suites passed. Its reconciliation explicitly concluded that continued local default/desktop execution added no deciding claim; the duplicate default run was interrupted and honestly retained as cancelled. Existing proportionality and verification rules were invoked to stop the duplication.

This supports earlier evidence-reuse decisions, not a new universal ban on local plus CI testing. Different environment obligations or genuinely independent observations can still justify both.

Duplicate execution reconciliation | Existing marginal-value rule

3. Evidence growth, gates and current state

D. Evidence preservation became a large review surface

PR59 adds 58,316 lines across 166 files. Of those, 56,404 lines (96.72%) are plan/evidence material. Approximately 482 additions are production/API integration, 1,080 are tests, and 350 are benchmark/configuration/admission material. Thirty JSON files account for 41,335 lines.

The warm-start directory contains 146 files and about 10.94 MB. Fifteen integration-era commits include eleven docs-prefixed commits. There are not widespread byte-identical duplicate files: only three empty stderr files share exact duplicate blobs. The stronger supported concern is repeated serialization and navigation burden. A successor review retains a 118,614-byte prior review inside a 124,089-byte document while 145 of 153 selected identities remain unchanged.

Preserve originals, failures and corrections. A compact current decision index can point to immutable evidence instead of copying full presentations. Volume alone does not justify deleting scientific records or imposing a document-count quota.

PR59 diff | Carried-forward review and reuse

E. Acceptance vocabulary hid different decisions

The earlier warm-start rule concerned repeatable adverse workloads; later current-main integration prospectively required an upper interval endpoint at or below zero and blocked inconclusive outcomes. The latest warm-OFF controls have intervals crossing zero. They do not demonstrate slowdown or benefit, but correctly fail that stronger rule. Historical warm-ON gains answer a different question.

The standards require an owned metric, workload, comparison, variability and consumer impact; they do not prescribe a zero-margin confidence bound. This is a project-owned acceptance-design issue, not evidence that the generic standards mandate the wrong statistic. Do not retroactively change the failed result to pass; keep measurement and release disposition separate.

Earlier rule | Later prospective rule | Existing threshold-authority principle

F. Historical truth and live status became fragmented

The composition ledger literally repeats one qualification block. Some preparation-time files retain pending statuses even after PR142/138/143 merged; later PR comments contain the final outcomes. Preserving historical status is legitimate, but readers lack a consistently discoverable current-disposition view. This is not evidence that owners abandoned the work.

Original block | Repeated block | Existing single-owner and link-not-copy guidance

4. Cross-project boundary failures

G. Mutable policy duplicated as literal text

A CI budget change from 30 to 60 minutes left a test expecting the old full header. PR142 and PR143 each encountered the same two errors in 219-case runs. The repair coordinated the prefix and rejection tests without changing numerical assertions. No evidence shows that standards required this exact-text representation.

The general lesson is one authority for mutable policy and tests of the owning semantic invariant. Exact representation checks remain appropriate when representation itself is the contract.

PR142 repair | PR143 recurrence

H. Transition ownership escaped steady-state checks

Rheon repeats a family of event and resource-boundary defects: coalesced node-pan input, brush release/new-press ownership, load-frame events replaying into a new document, receipt aliasing, queued commands bypassing Pending, and underestimated simultaneously live allocations. These are related boundary risks, not identical bugs.

The voxel ledger enumerates at least 25 implementation review findings across 12 component entries. Those are not 25 wasted reviews: they corrected numerical, ownership, aliasing and persistence defects. Its 63 plan/report files include 25 review, 12 admission and 14 verification documents; those counts measure surface area, not labor or waste.

Worker, load and adoption corrections | Late claim-to-observation reconciliation

I. Qualification clients violated real contracts

Tuldok provides independent recurrence: a wrong diagnostic prefix caused a completed fitting/validation attempt to fail downstream; server-only imports broke a pinned worker; base-only discovery ran 900 tests for 1,324 seconds before exposing optional-dependency assumptions; and multiple browser clients treated acknowledgements as semantic readiness.

Existing standards already require faithful early boundary observations and material environment preconditions. Make their applicability to test harnesses and handoffs unmistakable, rather than adding parser-, browser- or package-specific rules.

Late parser mismatch | Dependency matrix failure | Readiness race

J. Milestone closure was mistaken for objective closure

Tuldok explicitly corrected earlier completion accounting: bounded milestones did not finish the persistent objective. Later integration separated actual qualification dependencies from branch ancestry. This is already covered by scope and dependency principles. Keep the current claim, unresolved dependency and next action explicit; do not inherit stale blockers mechanically.

5. Minimal wording changes worth considering

These are proposed replacements or applicability clarifications, not findings that their absence demonstrably caused every incident. No edits have been made.

1. Remove a broad lower-level test obligation

Documentation says a local implementation change preserving documented decisions "requires tests and code review". Core permits adequate construction proof or existing evidence. Core precedence resolves the tension, and the sentence does not explicitly demand a new test, but its blanket wording is unnecessary.

Suggested replacement: "A local implementation change that preserves all documented decisions and contracts follows applicable verification and change-review requirements; it does not require an unrelated README or ADR edit."

Documentation target | Higher-precedence evidence rule

2. Bind findings to the subject actually observed

Extend the existing evidence-context paragraph, not a new protocol: "Bind each result or review finding to the material subject actually observed. Obtain its identity from the same stable observation, or detect intervening change before attributing the result."

This closes a precision gap illustrated by V1 findings carrying a V2 hash. It does not require hashes everywhere or a coordination service for read-only reviews. A stable revision, retained input, immutable buffer or detected intervening change may suffice.

Verification reporting target

3. Explicitly include the verification client

Replace the setup/action/observation/cleanup sentence with: "Make setup, action, observation, and cleanup reviewable, including the verification client's applicable lifecycle, dependency, and environment preconditions. Choose a structure suited to the claim and add explicit phases or comments when needed for clarity."

The deciding requirement remains the real claim, not a new universal test count. A successful command satisfies a claim only if the required observations and selected obligations actually executed.

Test-design target

4. Route proportionality through acceptance, too

Use one shared applicability condition in Router and Proportionality: "An additional investigation, evidence cycle, review, or re-plan could delay a sufficient implementation or acceptance decision." The current emphasis on delaying implementation can obscure post-implementation review/CI loops.

Router target | Proportionality applicability

6. Apply existing principles before adding more

Optional clarification: valid evidence at commit time

Commit says "Before every commit" and then "run" selected verification. This does not mandate full local-plus-CI duplication, but can be read as requiring repetition for an administrative/evidence commit.

Suggested step: "Confirm that the selected focused and affected verification remains valid for the staged slice. Run or rerun checks when relevant subject, inputs, environment, or gate requirements changed, or when an owned gate requires fresh execution." Preserve explicit fresh-execution gates.

Commit target

Operational changes, not new bureaucracy

Before expensive work, identify the unresolved claim and the cheapest faithful observation that could change its disposition. Use existing evidence when its subject and environmental obligations remain valid. Apply the same ownership and lifecycle principles to harnesses as to product clients. Review transition states and peak overlap, not only isolated steady states.

Separate immutable history from one current disposition. That can be a short section in an existing plan or issue; it does not require a new database, ledger schema or mandatory document. Link evidence rather than copying complete narratives. Explicitly separate implementation ancestry, qualification dependency and completion scope.

Validate changes proportionately

Use a small set of historical cases as semantic checks of the proposed wording: an unchanged evidence-only commit; V1 reviewed while V2 is written; a test client reading before semantic readiness; an optional-dependency environment; and an inconclusive result under an owned release rule. Ask whether the clarified principle selects the appropriate action without introducing extra required work.

Do not create brittle tests that assert the exact proposed prose. Check whether readers preserve the principle across unrelated examples, recognize when existing evidence is sufficient, and retain failed outcomes. One focused review of actual ambiguity is enough unless a new material risk emerges.

What to preserve

Keep the successful safeguards: independent review of consequential changes, original failed measurements, explicit uncertainty, source-bound claims, distinct native versus synthetic/hardware evidence, and no retroactive threshold relaxation. The audit found those controls catching real defects and preventing stronger unsupported claims.

Bottom line

The best-supported weakness is operational usability and consistency of the standards, not a shortage of principles. Make a few lower-level clauses harder to misread, improve applicability, and reduce competing representations. Do not turn each historical incident into another mandatory standard.

7. Source map and qualifications

All links below are pinned to inspected source revisions unless they intentionally identify a later PR discussion. They provide the deciding records behind this report.

Coding-Standards

Core; Content-bound review and current state; Marginal value and gate authority; Early faithful observation; Bounded investigation and stopping; Limits of literal and harness evidence; Concurrent-integration applicability.

Historical effectiveness findings register documents earlier checker coupling and procedural amplification. Historical findings are not assumed to remain current defects.

Rheon PR59

Execution ledger; Original failure, corrected results and scoped claims; Current-main qualification protocol; Driver findings; Attribution repair; Unadopted release-risk disposition proposal.

Rheon composition and voxel authoring

Composition ledger; Reviewer-approved evidence reuse; Voxel ledger; Bounded M5 acceptance; Later PR143 completion; Later integration-lead handoff.

Tuldok comparator

Completion-accounting correction; Semantic dependency separation; Pinned-worker dependency failure; Lifecycle/readiness recurrence; Successful reuse without repeating model execution.

Interpretation limits

Counts refer only to inspected snapshots and documented events. Review findings are not equivalent to independent test failures, elapsed command times are not total human effort, and file counts are not value metrics. Cause is not inferred from temporal sequence alone. No blanket performance tolerance, universal statistical threshold or project-specific tool mandate is recommended.

The standards audit inspected the current Core/Router and relevant planning, implementation, verification, proportionality, documentation, tooling, build, commit, performance, ownership and Rust/concurrent-integration material. It did not qualify the entire executable standards engine or prove all possible reader interpretations.